第四章 Lesson9
处理器架构：指令集与逻辑设计

笔记整理自 *2026* 年 *10* 月 *15* 日课程

第一部分讲**指令集体系结构（instruction set architecture, ISA）**，并定义一个教学用的指令集 Y86-64。第二部分转向**逻辑设计（logic design）**，从逻辑门讲到组合电路、寄存器、寄存器堆和存储器，最后把指令执行的顺序控制交给 HCL（Hardware Control Language，硬件控制语言）描述。下一讲会把这些构件拼成一个能真正执行 Y86-64 指令的处理器。

1. 指令集体系结构的位置

1.1. ISA 是软硬件之间的接口[1]

ISA 是处理器对上层软件承诺的全部东西，它包含三部分内容：

- 汇编语言视角下可见的**处理器状态（processor state）**：寄存器、内存、条件码、程序计数器；
- 一组**指令（instructions）**，例如 `addq`、`pushq`、`ret`，以及每条指令对状态的改变；
- 每条指令**如何编码成字节**，也就是机器码的格式。

ISA 位于一个多层抽象里。往上，编译器和操作系统把高级语言程序翻译成这个指令集能执行的代码序列；往下，微架构设计决定这些指令怎样被拆成数据通路上的动作，再往下是电路设计、芯片布局。同一套 ISA 可以有多种实现，只要对外行为一致，内部怎么搭是自由的。

1.2. Y86-64 与 x86-64 的关系

省流：x86 太麻烦复杂了，看看 y86 就够了。

原版： x86-64 指令集从 1978 年的 8086 一路累积而来，编码不规则、指令长度可变、寻址方式繁多，直接拿来设计处理器会把注意力从核心概念转移到历史包袱上。因此定义了一个精简版：Y86-64。它的**程序员可见状态**与 x86-64 基本一致，指令是 x86-64 的子集，只是编码规则化了，长度更容易判定。

最后对它的定位是：Y86-64 介于 CISC 和 RISC 之间[2]。它有条件码和变长的寄存器—内存操作数，这两点偏 CISC；它没有复杂寻址方式、没有栈式指令，这两点偏 RISC。

2. Y86-64 的程序员可见状态

Y86-64 把处理器状态分成五个部分，后面搭硬件时每一部分都对应一个具体部件。

- **程序寄存器（program registers）**：15 个 64 位寄存器，与 x86-64 同名同编号，只是省略了 `%r15`，所以 `%r8` 到 `%r14` 是能用的。
- **条件码（condition codes）**：三个一位标志。少了 `CF`（carry flag)。至于 CF 和 OF 分别代指什么，自己看看教材吧 hhh。
- **程序计数器（program counter, PC）**：下一条指令的地址。
- **程序状态（program status, Stat）**：表示正常执行还是出现了某种错误，取值见第 5 节。
- **内存（memory, DMEM）**：按字节编址的存储阵列，按**小端法**（little-endian）存放 64 位字，即低字节放在低地址。

[1]课上老师非常强调了抽象的概念和作用，但是同时需要考虑的是过度的抽象会带来硬件设备使用的不充分，所以精专与通用之间需要寻找一个平衡。很有生活哲理啊！生命也可以抽象：我们不一定有时间关心一日三餐是如何从田间走向灶头的(这会方便轻松许多)，但是忽略太多这样的细节也会让生命索然无味。

[2]之后会写。读作：sisk, risk

3. 指令编码

3.1. 编码规则

Y86-64 规定每条指令不超过 10 字节，且**从第一个字节就能唯一确定指令长度**。这是它区别于 x86-64 的地方之一。

编码的组织方式如下：

- 第一个字节分成两个 4 位域：高 4 位是**指令码（icode）**，低 4 位是**功能码（ifun）**。同一类操作共享 icode，具体是哪一条由 ifun 区分。
- 需要寄存器操作数的指令，第二字节高 4 位放源寄存器编号，低 4 位放目的寄存器编号。
- 需要立即数或位移量时，紧跟一个或两个 8 字节的补码整数，按小端序存放。

寄存器编号沿用 x86-64：`%rax` 是 0，`%rcx` 是 1，`%rdx` 是 2，`%rbx` 是 3，`%rsp` 是 4，`%rbp` 是 5，`%rsi` 是 6，`%rdi` 是 7，`%r8` 到 `%r14` 依次是 8 到 14（写成十六进制 8 到 E）。编号 `0xF` 表示「没有寄存器」。用 4 位编码寄存器，硬件上就是一个 16 选 1 的译码，`0xF` 这个空位在后面控制逻辑里会被反复用到，用来表示「这一路不需要读或不需要写」。

编号	寄存器	编号	寄存器
0	`%rax`	8	`%r8`
1	`%rcx`	9	`%r9`
2	`%rdx`	A	`%r10`
3	`%rbx`	B	`%r11`
4	`%rsp`	C	`%r12`
5	`%rbp`	D	`%r13`
6	`%rsi`	E	`%r14`
7	`%rdi`	F	无寄存器

由于编码规整，指令长度只取决于 icode：`halt`、`nop`、`ret` 是 1 字节；带两个寄存器域的是 2 字节；带一个 8 字节常数的 `irmovq`、`rmmovq`、`mrmovq` 是 10 字节；带一个 8 字节目的地址的 `jXX` 和 `call` 是 9 字节。

3.2. 完整指令表

汇编形式	字节数	编码	说明
halt	1	`0 0`	停止执行
nop	1	`1 0`	空操作
rrmovq rA, rB	2	`2 0 rA rB`	寄存器到寄存器
cmovXX rA, rB	2	`2 fn rA rB`	条件传送，fn 见下表
irmovq V, rB	10	`3 0 F rB V`	立即数到寄存器
rmmovq rA, D(rB)	10	`4 0 rA rB D`	寄存器到内存
mrmovq D(rB), rA	10	`5 0 rA rB D`	内存到寄存器
OPq rA, rB	2	`6 fn rA rB`	算术逻辑运算
jXX Dest	9	`7 fn Dest`	条件跳转

汇编形式	字节数	编码	说明
call Dest	9	`8 0 Dest`	过程调用
ret	1	`9 0`	从过程返回
pushq rA	A	`A 0 rA F`	压栈
popq rA	B	`B 0 rA F`	出栈

`fn` 的取值有三组，都从 0 开始连续编号：

fn	OPq	cmovXX / jXX
0	addq	rrmovq / jmp
1	subq	cmovle / jle
2	andq	cmovl / jl
3	xorq	cmove / je
4	—	cmovne / jne
5	—	cmovge / jge
6	—	cmovg / jg

编码里有两个小设计值得注意。第一，`pushq` 和 `popq` 的第二个寄存器域填 `F`，因为它们只涉及一个寄存器，另一个位置空着。第二，`halt` 的编码是 `00`，也就是全零字节。一段被清零的内存若被误当成指令执行，解码出来正好是 `halt`，程序会停下来而不是执行一串无意义的字节。

4. 各条指令的语义

4.1. 算术与逻辑运算

四条整数运算 `addq`、`subq`、`andq`、`xorq` 统称 `OPq`，编码只有 fn 不同。语义是 `rB = rB op rA`，即第二个操作数既是源也是目的，结果写回 `rB`，并设置条件码。

`addq %rax, %rsi` 的编码是 `60 06`：第一字节 `6` 是 OPq 的 icode，`0` 是 addq 的 fn；第二字节高半字节 `0` 是 `%rax`，低半字节 `6` 是 `%rsi`。

与 x86-64 的一个差别是：Y86-64 的算术指令只能对寄存器做运算，操作数不能来自内存，也不带立即数。想给寄存器加一个常数，必须先 `irmovq` 把常数装进另一个寄存器。

4.2. 数据传送

四种传送按「从哪里到哪里」区分：

- `rrmovq rA, rB`：寄存器到寄存器，语义 `rB = rA`，就是无条件传送。
- `irmovq V, rB`：立即数到寄存器，语义 `rB = V`。第二个字节的高半字节固定为 `F`，因为源不是寄存器。
- `rmmovq rA, D(rB)`：寄存器到内存，语义 `M[rB + D] = rA`，源在 rA，目的地址由 rB 加位移量得到。
- `mrmovq D(rB), rA`：内存到寄存器，语义 `rA = M[rB + D]`，方向反过来。

四者的共同点是地址计算只有「基址寄存器加 8 字节位移量」这一种形式，没有比例变址，没有 `leaq`。

几个编码例子，都可以按上面的规则手工推出来：

```
irmovq $0xabcd, %rdx      30 f2 cd ab 00 00 00 00 00 00
rrmovq %rsp, %rbx         20 43
```

```
mrmovq -12(%rbp), %rcx    50 15 f4 ff ff ff ff ff ff ff
rmmovq %rsi, 0x41c(%rsp)  40 64 1c 04 00 00 00 00 00 00
```

以最后一个为例：4 是 icode，第二字节 6 是源 `%rsi`、4 是基址 `%rsp`，最后 8 字节是位移量 `0x41c` 的小端表示。`mrmovq` 那一条的第二字节 `15` 里，`1` 是目的 `%rcx`，`5` 是基址 `%rbp`，位移量 −12 的补码是 `f4 ff ff ff ff ff ff ff`。

4.3. 条件传送

`rrmovq` 加上一个「什么时候搬」的条件，就得到一组 `cmovXX`：`cmovle`、`cmovl`、`cmove`、`cmovne`、`cmovge`、`cmovg`。它们根据条件码决定是否执行 `rB = rA`：条件成立就搬，不成立就当作没发生，`rB` 保持原值。条件码本身不被修改。

这条指令的意义在于把分支变成数据选择。C 语言里 `if (x < y) x = y;` 可以编译成一次比较加一条 `cmovl`，不必出现跳转，也就不必为跳转预测付出代价。

4.4. 跳转

`jXX Dest` 与 x86-64 的同类指令语义相同，按条件码决定是否把 PC 改成 `Dest`；`jmp` 无条件跳转。编码上的差别在这里：Y86-64 直接**编码绝对地址**，占 8 字节，而 x86-64 用相对当前 PC 的偏移量。绝对地址意味着汇编器必须知道目标指令最终落在哪个地址，所以汇编过程要多做一次地址回填；好处是解码时不必再加 PC。

4.5. 栈操作

栈在内存中从高地址向低地址增长，`%rsp` 指向栈顶元素，栈顶是栈内地址最低的那个位置。

`pushq rA` 的动作是先把 `%rsp` 减 8，再把 `rA` 写入 `M[%rsp]`；`popq rA` 先把 `M[%rsp]` 读进 `rA`，再把 `%rsp` 加 8。这两条指令与 x86-64 的行为一致。

4.6. 过程调用与返回

`call Dest` 把**下一条指令的地址**压栈，然后跳到 `Dest`；`ret` 从栈顶弹出 8 字节并作为下一条指令的地址。这两条指令加上栈，就构成了过程调用所需的控制传递机制。参数怎么传、哪些寄存器由被调用者保存，仍然由调用约定决定，指令集本身不规定。

4.7. 空操作与停机

`nop` 什么也不做，常用于对齐或占位。`halt` 停止执行，在模拟器里就是结束程序的信号。x86-64 也有类似指令，但用户态程序不能执行它，需要陷入内核，而 Y86-64 允许直接在用户态停机。

5. 状态码

`Stat` 描述程序当前处于什么状态，取值如下：[3]

助记符	编码	含义
AOK	1	正常执行
HLT	2	遇到 `halt` 指令
ADR	3	访问了非法地址（取指或读写数据）
INS	4	遇到非法指令

期望行为很简单：状态是 `AOK` 就继续取指执行，否则停止整个程序。这条规则让模拟器和后面的硬件都有一个统一的异常出口，不必为每种错误单独设计流程。

[3] AOK:啊！OK！HLT：halt，ADR:下次拼写 address 要记得两个 d。

6. CISC 与 RISC

6.1. CISC：复杂指令集

CISC（complex instruction set computer，复杂指令集计算机）的代表是 IA32，它的设计哲学是「把常见的编程任务做成一条指令」，用指令数量换程序长度。

- 指令集以栈为中心：用栈传递参数、保存返回地址，有显式的 `push` 和 `pop`。
- 算术指令可以直接访问内存，例如 `addq %rax, 12(%rbx,%rcx,8)` 一次完成读内存、加法、写内存。
- 寻址方式复杂，一条指令里可以同时出现基址、变址、比例因子、位移量。
- 有条件码，由算术和逻辑指令顺带设置。

对编译器的好处是目标代码短，一段 C 程序对应的字节数少。

6.2. RISC：精简指令集

RISC（reduced instruction set computer，精简指令集计算机）最早是 IBM 的内部项目，后来由 Hennessy（斯坦福）和 Patterson（伯克利）推广开来。它的思路相反：指令少而规整，让硬件小而快，需要更多指令才能完成同样的任务，但每条指令都能在很短的时间内完成。

- 面向寄存器：寄存器多得多，典型是 32 个，参数、返回值、临时值都放寄存器。
- 只有 load 和 store 两条（类）指令能访问内存，对应 Y86-64 的 `mrmovq` 和 `rmmovq`。
- 没有条件码。比较指令把结果 0 或 1 写进寄存器，后面的条件跳转直接读这个寄存器。
- 指令格式固定成少数几种，编码字段位置一致，解码简单。

6.3. 争论与现状

最初的争论很激烈。CISC 一方强调编译器好写、代码字节少；RISC 一方强调对优化编译器友好、简单芯片能跑得快。

现在的结论是：在桌面和服务器上，ISA 的选择已经不是技术问题。硬件足够多时，任何指令集都能跑得很快，可执行文件兼容性反而更重要，所以 x86-64 保留了下来，同时吸收了大量 RISC 特征，例如寄存器数量增加到 16 个并用寄存器传参。在嵌入式领域，RISC 仍然占优，因为芯片更小、更便宜、功耗更低，绝大多数手机用 ARM。

对这门课来说，真正要带走的判断是：ISA 设计的重要性在下降，实现技术的重要性在上升。

7. 逻辑设计的三个基本问题

进入第二部分。任何硬件都要解决三件事：

- **通信（communication）**：值怎样从一个地方传到另一个地方。硬件上就是一根导线，用低电平表示 0、高电平表示 1。
- **计算（computation）**：怎样把输入变成输出。硬件上就是计算布尔函数。
- **存储（storage）**：怎样把一位信息留住。硬件上就是能自保持的电路。

这三件事对应三类构件：导线、逻辑门组成的组合电路、由时钟驱动的存储元件。处理器的数据通路就是把它们按指令语义连起来。

7.1. 数字信号

真实导线上的电压是连续变化的，但电路只关心它落在哪个区间。做法是设定两个阈值：高于上限算 1，低于下限算 0，中间留一段保护带。只要噪声不把电压推出所属区间，逻辑值就不受影响。这个约定带来的是工程上的简化：电路可以做得简单、小、快，不必追求精确的模拟量。

7.2. 逻辑门

逻辑门是计算的基本单元，输出是输入的布尔函数。三种基本门是「与」(and)、「或」(or)、「非」(not)，分别写作 `a && b`、`a || b`、`!a`。

门的输出**连续地**跟随输入变化，只是有延迟：输入变化后，输出经过一小段时间才稳定，上升沿和下降沿的延迟还不一样。这一点在后面分析电路时才用得上，组合电路的输出总是在输入变化后的一小段时间内变成正确值。

8. 组合逻辑与 HCL

8.1. 组合电路

组合电路（combinational circuit）是无环的逻辑门网络：把若干门的输出接到其他门的输入，不允许出现回路。它的性质是：只要输入保持稳定，经过一段延迟后输出稳定为输入的布尔函数值；输入一变，输出跟着变。

「无环」这个限制不是形式上的洁癖。有环的网络会把输出接回自己的输入，输出就不再由输入单独决定，而要由历史状态决定，那已经属于存储元件的范畴。第 11 节讲锁存器时会看到，存储正是靠一个环实现的。

8.2. 用 HCL 描述硬件

HCL（Hardware Control Language，硬件控制语言）是一个非常小的硬件描述语言，只用来描述要修改的那部分控制逻辑。它有两种数据类型：`bool` 表示一位布尔值，`int` 表示一个字（不规定位宽，可以是 64 位，也可以是 1 位）。语句只有两种形式：

```
bool a = bool-expr;
int  A = int-expr;
```

与 C 的区别在于语义：HCL 的赋值不是「把值存起来」，而是「给这条线起个名字，它恒等于右边表达式的值」。所有语句同时成立，没有先后顺序。

8.3. 位相等与字相等

判断两位是否相等，逻辑表达式是

```
bool eq = (a && b) || (!a && !b);
```

它的含义是「同时为 1，或者同时为 0」。

要判断两个字是否相等，就把 64 个这样的比较器并排摆开，再把 64 个结果与起来：只有每一位都相等，输出才为 1。

```
bool Eq = (A == B);
```

`==` 是 HCL 的字级运算，硬件上就是上面那棵「64 个位比较器 + 与树」。写 HCL 时不必展开，工具会负责生成电路，这正是用语言描述硬件的好处。

8.4. 多路选择器

多位选择器（multiplexor, MUX）的功能是：控制信号 `s` 为 1 时输出 `a`，为 0 时输出 `b`。位级写法就是

```
bool out = (s && a) || (!s && b);
```

字级多路选择器一次选整个字，64 个位级 MUX 并排，共用同一个 `s`。HCL 用**选择表达式（case expression）**描述：

```
int Out = [
    s : A;
    1 : B;
];
```

方括号里是一串「测试 : 值」对，按顺序求值，返回**第一个**测试为真的那一项的值。最后一项写 `1`，保证总有匹配。

这个「按顺序、取第一个匹配」的语义很重要：后面的测试可以假设前面的都不成立，从而省略一部分条件。

8.5. 两个字级例子

求三个字的最小值：

```
int Min3 = [
    A < B && A < C : A;
    B < A && B < C : B;
    1 : C;
];
```

前两项都显式写出了完整条件，因为如果不写，第一项不成立并不说明 `A` 不是最小。最后一项用 `1` 兜底。

四选一多路选择器用两个控制位 `s1`、`s0` 选四个输入。利用顺序匹配，测试可以简化：

```
int Out4 = [
    !s1 && !s0 : D0;
    !s1        : D1;
    !s0        : D2;
    1          : D3;
];
```

第二项只测 `!s1`：能走到这一行，说明第一行已经不成立，也就是 `s0` 必为 `1`，所以 `!s1` 成立就意味着 `(s1,s0) = (0,1)`。第四行同理。

8.6. 算术逻辑单元

ALU（arithmetic/logic unit，算术逻辑单元）是组合电路，控制信号选择它做哪一种运算。Y86-64 需要四种：加法、减法、与、异或，正好对应 `OPq` 的四个 fn。同时 ALU 还要顺带算出三个条件码 `ZF`、`SF`、`OF`，送给后面要讲的标志寄存器。

ALU 是组合逻辑，所以它**连续**响应输入：`X` 和 `Y` 一变，输出 `valE` 和条件码就立刻跟着变，不需要时钟。这一点在数据通路里会引起一个麻烦：条件码必须在时钟边沿被保存下来，否则它们会随着输入变化而闪变。

9. 时序逻辑：寄存器、寄存器堆、存储器

9.1. 寄存器

这里的「寄存器」指硬件上存一个字的元件，和汇编里说的程序寄存器不是同一个层次的概念。程序寄存器们是用这种元件搭出来的。

一个 64 位寄存器由 64 个一位存储单元组成，每个单元在时钟上升沿（rising edge）把输入装进来，其他时间输入和输出之间被隔断。所以寄存器的行为可以概括成两句话：平时它是一堵墙，输入变了输出不变；时钟上升沿时它打开一下，把输入的值装进来。

这个「只在边沿动作」的性质是整个同步设计的基石。它保证一个时钟周期内发生的所有组合逻辑变化不会互相影响，只有边沿时刻的值被锁定。

9.2. 状态机：累加器

把组合逻辑和寄存器接成环，就有了状态。累加器：每个周期把输入加到累加值上，除非 `Load` 信号要求重新开始。

- 组合部分：一个 ALU 做加法，一个 MUX 在「`0`」和「当前累加值」之间按 `Load` 选择；
- 存储部分：一个寄存器保存累加结果；
- 每个时钟上升沿，寄存器装入 `ALU(输入, MUX 的输出)`。

时序就能读出来：`Load` 为 1 的周期，MUX 输出 0，寄存器装入输入本身，累加重新开始；`Load` 为 0 的周期，寄存器装入旧值加输入，累加继续。图上 `x0, x0+x1, x0+x1+x2` 然后跳到 `x3`，正是 `Load` 在两个周期里为 1 的结果。

9.3. 寄存器堆

处理器的寄存器堆（register file）存放所有程序寄存器的值，`%rax`、`%rsp` 等都在里面。它与单个寄存器的差别在于：用编号当地址，按地址读出或写入。

- 有两个读端口（read port）`srcA`、`srcB`，各自给出地址、各自输出数据。这样一条 `OPq` 指令可以一次读出两个操作数。
- 有一个写端口（write port）`dstW`，给出地址和数据。
- 编号 `0xF` 表示不读也不写。这正好利用了「没有寄存器」这个编码空位：一条 `irmovq` 不需要读任何寄存器，就把两个读端口的地址都设成 `0xF`。

读写时序不对称，这一点必须记清：

- **读**像组合逻辑。地址一变，经过一段延迟，输出就变成对应寄存器的值。地址保持不变，读出的值也保持。
- **写**像寄存器。只有时钟上升沿才真正写入。

因此在一个周期内，可以先按旧地址读出旧值，边沿到来时再把新值写回同一个寄存器，读和写互不干扰。这个性质在实现 `addq %rax, %rax` 这类指令时是必需的。

9.4. 存储器

RAM（random-access memory，随机访问存储器）存放多个字，地址输入决定读写哪一个。读的时候给出地址，输出就在对应位置的数据；写的时候在时钟上升沿把数据写进指定地址。处理器的指令存储器和数据存储器都按这个方式工作。

10. HCL 语法小结

HCL 的表达式按返回值类型分成两类。

返回布尔值的运算：

- 逻辑运算：`a && b`、`a || b`、`!a`；
- 字比较：`A == B`、`A != B`、`A < B`、`A <= B`、`A >= B`、`A > B`；
- 集合成员判断：`A in { B, C, D }`，等价于 `A == B || A == C || A == D`。译码逻辑里常用它把若干 icode 归成同一类。

返回字的表达式：

- 选择表达式 `[ 测试 : 值; ... ]`，按顺序求值，返回第一个成立的测试对应的值。所有测试都不成立时行为未定义，所以最后一项通常写 `1`。

两个容易混淆的地方。第一，`&&`、`||` 在这里没有短路求值的概念，它们描述的是门电路，所有输入都要参与计算。第二，`int` 不带位宽，同一个名字在不同位置可以代表不同宽度的字，位宽由使用它的电路决定。